Introduccion

Orden de desarrollo

Archivo	Carpeta	Quien lo necesita
Pago.java	src/models/	Todos los demas archivos
Dano.java	src/models/	DanoService, DanosPanel
Mantenimiento.java	src/models/	MantenimientoService, Panel
PagoService.java	src/services/	PagosPanel
DanoService.java	src/services/	DanosPanel
MantenimientoService.java	src/services/	MantenimientoPanel
PagosPanel.java	src/menu/	MainWindow
DanosPanel.java	src/menu/	MainWindow
MantenimientoPanel.java	src/menu/	MainWindow

1. Pago.java

Es la clase que representa un pago en el sistema. Como si fuera un recibo digital con toda la informacion del pago.

Paso 1 — Crear la clase

- Clic derecho en la carpeta src/models/
- New → Java Class
- Escribe el nombre: Pago
- Enter

Paso 2 — Agregar el import

Como el campo fecha usa el tipo Date de la base de datos, necesitas importarlo al inicio del archivo, antes de la palabra public class:

```
import java.sql.Date;
```

Paso 3 — Definir los atributos

Los atributos son las caracteristicas del pago. Son privados para que nadie los modifique directamente desde afuera de la clase:

Atributo	Tipo	Descripcion
id	int	Numero unico del pago, se genera automatico en la BD
reservaId	int	A que reserva pertenece este pago
monto	double	Cuanto se pago en RD$
metodoPago	String	EFECTIVO, TARJETA o TRANSFERENCIA
fecha	Date	Cuando se realizo el pago (java.sql.Date)
notas	String	Observaciones adicionales, puede estar vacio

Paso 4 — Generar el constructor vacio

El constructor vacio es necesario para crear un objeto Pago sin datos todavia. IntelliJ lo genera automaticamente:

- Clic derecho dentro de la clase → Generate
- Selecciona Constructor

- No selecciones ningun campo → OK

Paso 5 — Generar el constructor completo

El constructor completo es para crear un Pago con todos los datos de una sola vez:

- Clic derecho dentro de la clase → Generate
- Selecciona Constructor
- Selecciona TODOS los campos → OK

Nota: El 'this.' que ves en el codigo generado significa: el atributo de ESTA clase. Lo diferencia del parametro que llega.

Paso 6 — Generar Getters y Setters

Los getters sirven para leer el valor de un atributo. Los setters para cambiarlo. IntelliJ los genera automaticamente:

- Clic derecho dentro de la clase → Generate
- Selecciona Getters and Setters
- Selecciona TODOS los campos → OK

Paso 7 — Generar toString()

El toString sirve para cuando quieras imprimir un objeto Pago y ver sus datos facilmente. Es util para encontrar errores:

- Clic derecho dentro de la clase → Generate
- Selecciona toString()
- Selecciona los campos: id, reservaId, monto, metodoPago, fecha → OK

Nota: Guarda el archivo con Ctrl+S cuando termines.

2. Dano.java

Representa un dano reportado en un vehiculo. Guarda que vehiculo fue dañado, que cliente fue responsable, cuanto cuesta repararlo y si ya fue cobrado.

Crear la clase

- Clic derecho en src/models/ → New → Java Class → Dano → Enter
- Importar al inicio: import java.sql.Date;

Atributos

Atributo	Tipo	Descripcion
id	int	Numero unico del dano
vehiculoId	int	A que vehiculo le paso el dano
clienteId	int	Que cliente fue el responsable
descripcion	String	Que fue lo que paso exactamente
costo	double	Cuanto cuesta reparar el dano en RD$
fecha	Date	Cuando ocurrio el dano (java.sql.Date)
estado	String	Solo puede ser: PENDIENTE o COBRADO

Generar todo automaticamente

- Constructor vacio: Generate → Constructor → sin campos → OK
- Constructor completo: Generate → Constructor → todos los campos → OK
- Getters y Setters: Generate → Getters and Setters → todos → OK
- toString: Generate → toString → id, vehiculoId, descripcion, estado → OK

Nota: Guarda con Ctrl+S cuando termines.

3. Mantenimiento.java

Representa un mantenimiento realizado a un vehiculo. Guarda que tipo de mantenimiento fue, el kilometraje cuando se hizo y cuantos kilometros faltan para el proximo.

Crear la clase

- Clic derecho en src/models/ → New → Java Class → Mantenimiento → Enter
- Importar al inicio: import java.sql.Date;

Atributos

Atributo	Tipo	Descripcion
id	int	Numero unico del mantenimiento
vehiculoId	int	A que vehiculo se le hizo el mantenimiento
tipo	String	Que tipo fue: Aceite, Frenos, Llantas, General, Otro
km	int	Cuantos km tenia el vehiculo cuando se hizo
proximoKm	int	A cuantos km toca hacer el proximo mantenimiento
costo	double	Cuanto costo el mantenimiento en RD$
fecha	Date	Cuando se realizo (java.sql.Date)
notas	String	Observaciones adicionales

Generar todo automaticamente

- Constructor vacio: Generate → Constructor → sin campos → OK
- Constructor completo: Generate → Constructor → todos los campos → OK
- Getters y Setters: Generate → Getters and Setters → todos → OK
- toString: Generate → toString → id, vehiculoId, tipo, km, fecha → OK

Nota: Guarda con Ctrl+S cuando termines.

4. PagoService.java

El service es la clase que conecta la pantalla con la base de datos. Usa los DAOs que ya hizo Luis Fernando. No escribe SQL directamente, solo llama los metodos del DAO.

Crear la clase

- Clic derecho en src/services/ → New → Java Class → PagoService → Enter

Que va adentro

Primero crea un objeto del DAO de Luis Fernando arriba de todo:

```
private PagoDAO pagoDAO = new PagoDAO();
```

Luego crea estos 4 metodos:

Atributo	Tipo	Descripcion
registrarPago(Pago p)	void	Llama a pagoDAO.insertar(p) para guardar el pago
listarPagos()	List<Pago>	Llama a pagoDAO.listarTodos() y retorna la lista
eliminarPago(int id)	void	Llama a pagoDAO.eliminar(id) para borrarlo
buscarPorReserva(int rese	vaLlidst)<Pago>	Retorna pagos de una reserva especifica

Nota: Cada metodo del service solo llama al metodo correspondiente del DAO. No escribas SQL aqui.

5. DanoService.java

Igual que PagoService pero para danos. Usa el DanoDAO de Luis Fernando.

Crear la clase

- Clic derecho en src/services/ → New → Java Class → DanoService → Enter

Crea el DAO arriba de todo:

```
private DanoDAO danoDAO = new DanoDAO();
```

Metodos que debe tener

Atributo	Tipo	Descripcion
registrarDano(Dano d)	void	Guarda un dano nuevo llamando al DAO
listarDanos()	List<Dano>	Retorna todos los danos
listarPendientes()	List<Dano>	Retorna solo los danos con estado PENDIENTE
cobrarDano(int id)	void	Cambia el estado del dano a COBRADO
eliminarDano(int id)	void	Borra un dano por su ID

Importante: El metodo listarPendientes() filtra los danos cuyo estado sea igual a PENDIENTE. Puedes usar un bucle o pedirle al DAO que lo filtre desde la base de datos.

6. MantenimientoService.java

Igual que los anteriores pero para mantenimientos. Usa el MantenimientoDAO de Luis Fernando.

Crear la clase

- Clic derecho en src/services/ → New → Java Class → MantenimientoService → Enter

Crea el DAO arriba de todo:

```
private MantenimientoDAO mantDAO = new MantenimientoDAO();
```

Metodos que debe tener

Atributo	Tipo	Descripcion
registrarMantenimiento(Mantenimiento m)	void	Guarda un mantenimiento nuevo
listarMantenimientos()	List<Mantenimiento>	Retorna todos los mantenimientos
listarPorVehiculo(int vehiculo)	List<Mantenimiento>	Retorna los de un vehiculo especifico
eliminarMantenimiento(int id)	void	Borra un mantenimiento por ID

7, 8 y 9. Los Paneles (Pantallas)

Los paneles son las pantallas que ve el usuario. Se hacen al final porque necesitan que los modelos y los services ya esten listos.

Importante: Haz los paneles SOLO cuando los 6 archivos anteriores esten completos y sin errores.

Orden de los paneles

- PagosPanel.java → DanosPanel.java → MantenimientoPanel.java

Que tiene cada panel

Cada panel tiene exactamente estas 4 partes:

Atributo	Tipo	Descripcion
Formulario (arriba)	JPanel	Campos de texto para ingresar los datos
Tabla (abajo)	JTable	Muestra todos los registros de la BD
Botones	JButton	Guardar, Limpiar, Eliminar
Service (arriba todo)	Objeto	Usa el Service, NUNCA el DAO directamente

Como crear el panel

- Clic derecho en src/menu/ → New → Java Class → PagosPanel → Enter
- La clase extiende de JPanel: public class PagosPanel extends JPanel
- Arriba crea el service: private PagoService pagoService = new PagoService()
- En el constructor crea el formulario, la tabla y los botones
- El boton Guardar toma los datos del formulario y llama a pagoService.registrarPago()
- La tabla se llena llamando a pagoService.listarPagos()

Nota: Para ver ejemplos de como hacer el JPanel con formulario y tabla, mira como Luis Fernando hizo el MainWindow o como Eze hizo el FlotaPanel.

Resumen Final

Tienes que entregar estos 9 archivos en este orden exacto:

#	Archivo	Carpeta	Estado
1	Pago.java	src/models/	Empieza aqui
2	Dano.java	src/models/	Segundo
3	Mantenimiento.java	src/models/	Tercero
4	PagoService.java	src/services/	Cuarto
5	DanoService.java	src/services/	Quinto
6	MantenimientoService.java	src/services/	Sexto
7	PagosPanel.java	src/menu/	Al final
8	DanosPanel.java	src/menu/	Al final
9	MantenimientoPanel.java	src/menu/	Al final